

MI WLAN API

1 Overview

This module (wireless local area network) provides a simple wifi signal scan. The connection function provides support for AP and STA modes.

2. API Reference

2.1. Function Module API

API name	Features
MI_WLAN_Init [#22-mi_wlan_init]	Wi-Fi device initialization
MI_WLAN_DeInit [#23-mi_wlan_deinit]	Wi-Fi device logout
MI_WLAN_Open [#24-mi_wlan_open]	Enable WLAN device and set working parameters
MI_WLAN_Close [#25-mi_wlan_close]	Turn off the Wi-Fi device.
MI_WLAN_Connect [#26-mi_wlan_connect]	Connect to WIFI hotspot or start AP working mode
MI_WLAN_Disconnect [#27-mi_wlan_disconnect]	Disconnect the WIFI hotspot or turn off AP mode
MI_WLAN_Scan [#28-mi_wlan_scan]	Scan for available hotspots
MI_WLAN_GetStatus [#29-mi_wlan_getstatus]	Get connection status
MI_WLAN_GetWlanChipVersion [#210-mi_wlan_getwlanchipversion]	Obtain the version number of the WLAN device FW

2.2. MI_WLAN_Init

- Features

Wi-Fi device initialization

- grammar

```
MI_RESULT MI_WLAN_Init(MI_WLAN_InitParams_t *pstInitParams);
```

- formal parameter

parameter name	describe	input Output
pstInitParams	Wlan device initialization parameter pointer	enter

- return value
 - 0 success.
 - Non-zero failure, refer to the [error code](#) [#jump] .
- rely
 - Header file: mi_wlan.h mi_wlan_datatype.h
 - Libraries: libmi_wlan libcjson libcrypto libssl libnl
 - Executable: iwlist wpa_cli wpa_supplicant
- Notice
 - The initialization of the Wlan device depends on a json configuration file, the public version is called wlan.json, located at `/config/wifi/wlan.json` or `/customer/wifi/wlan.json`.
 - This parameter currently does not support passing NULL.
 - 因为WLAN模块的可配置项目多，项目类型复杂，而且有不不确定性，所以引入一个json配置文件，该文件的内容以及配置项介绍，请参考 [APPENDIX A](#) [#jump1] 。
- 举例

下面的代码实现设置WLAN 设备初始化。

```
MI_WLAN_InitParams_t stParm = {"/config/wifi/wlan.json"};
MI_WLAN_Init(&stParm);
```

2.3. MI_WLAN_DeInit

- 功能

注销WLAN设备。
- 语法

```
MI_RESULT MI_WLAN_DeInit(void);
```

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_wlan.h mi_wlan_datatype.h
 - 库: libmi_wlan libcjson libcrypto libssl libnl
 - 可执行档: iwlist wpa_cli wpa_supplicant
- 举例

```
MI_RESULT ret;  
  
ret = MI_WLAN_DeInit();  
  
if(MI_SUCCESS != ret)  
{  
    return ret;  
}
```

2.4. MI_WLAN_Open

- 功能

使能WLAN设备，设置工作参数。
- 语法

```
MI_RESULT MI_WLAN_Open(MI_WLAN_OpenParams_t *pstParam);
```

- 形参

参数名称	描述	输入/输出
pstParam	Wlan设备的工作参数指针	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。
- 依赖
 - 头文件: mi_wlan.h mi_wlan_datatype.h
 - 库: libmi_wlan libbson libcrypto libssl libnl
 - 可执行档: iwlist wpa_cli wpa_supplicant
- 注意
 - 需要WLAN 已经初始化
- 举例

请参见[MI_WLAN_OpenParams_t](#) [#39_mi_wlan_openparams_t]的举例。

2.5. MI_WLAN_Close

- 功能
关闭WLAN 设备。
- 语法

```
MI_RESULT MI_WLAN_Close(MI_WLAN_OpenParams_t *pstParam);
```

- 形参

参数名称	描述	输入/输出
pstParam	Wlan设备的工作参数指针	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_wlan.h mi_wlan_datatype.h
 - 库: libmi_wlan libcjson libcrypto libssl libnl
 - 可执行档: iwlist wpa_cli wpa_supplicant
- 注意
 - 需要WLAN 已经初始化
- 举例

```
MI_RESULT ret;

MI_WLAN_Open_Params_t g_stOpenParam=
{E_MI_WLAN_ENTWORKTYPE_INFRA};

ret = MI_WLAN_Close (&g_stOpenParam);

if(MI_SUCCESS != ret)

{
    printf(" wlan close fail\n");

    return ret;
}
```

2.6. MI_WLAN_Connect

- 功能

建立wifi连接服务
- 语法

```
MI_RESULT MI_WLAN_Connect(WLAN_HANDLE *hWlan,
MI_WLAN_ConnectParam_t *pstConnectParam);
```

• 形参

参数名称	描述	输入/输出
hWlan	Wlan 句柄指针 1. (<0)表示要建立一个新sta连接，函数执行成功，会被赋予新连接的句柄。(sta mode) 2. (>0)表示是一个已经存在的连接。 3. (==WLAN_HANDLE_AP)表示ap mode。	输入/输出
pstConnectParam	Wifi连接的参数设定指针 详见 MI_WLAN_ConnectParam_t [#310-mi_wlan_connectparam_t]	输入

• 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

• 依赖

- 头文件: mi_wlan.h mi_wlan_datatype.h
- 库: libmi_wlan libcjson libcrypto libssl libnl
- 可执行档: iwlist wpa_cli wpa_supplicant

• 注意

- 如果wifi热点没有设置密码，需要传入一个空字符串 ""
- 目前超时设定没有正确生效，连接的超时由wlan.json 中，connect:infra:dhcp -T -t 参数给出详见[APPENDIX A](#) [#jump1]

• 举例

```
WLAN_HANDLE wlanHdl = -1; //std mode

MI_WLAN_ConnectParam_t stConnectParam = \
{E_MI_WLAN_SECURITY_WPA2, "123456", "", 5000};

MI_WLAN_Connect(&wlanHdl, &stConnectParam);
```

.....

2.7. MI_WLAN_Disconnect

- 功能

断开wifi连接服务

- 语法

```
MI_RESULT MI_WLAN_Disconnect(WLAN_HANDLE hWlan);
```

- 形参

参数名称	描述	输入/输出
hWlan	Wlan 句柄	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [jump]。

- 依赖

- 头文件: mi_wlan.h mi_wlan_datatype.h
- 库: libmi_wlan libcjson libcrypto libssl libnl
- 可执行档: iwlist wpa_cli wpa_supplicant

- 注意

确保hWlan>0

- 举例

```
WLAN_HANDLE wlanHdl = -1;

MI_WLAN_ConnectParam_t stConnectParam = \
{E_MI_WLAN_SECURITY_WPA2, "123456", "", 5000};

MI_WLAN_Connect(&wlanHdl, &stConnectParam);

MI_WLAN_Disconnect(wlanHdl);
```

.....

2.8. MI_WLAN_Scan

- 功能

扫描当前可用wifi连接

- 语法

```
MI_RESULT MI_WLAN_Scan(MI_WLAN_ScanParam_t *pstParam,  
MI_WLAN_ScanResult_t *pstResult);
```

- 形参

参数名称	描述	输入/输出
pstParam	扫描参数指针 目前没有实际使用	输入
pstResult	扫描到的当前可用wifi热点的信息指针	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [jump]。

- 依赖

- 头文件: mi_wlan.h mi_wlan_datatype.h
- 库: libmi_wlan libbson libcrypto libssl libnl
- 可执行档: iwlist wpa_cli wpa_supplicant

- 注意

输入参数pstResult中经过调用MI_WLAN_Scan，其中u8APNumber默认为最大值MI_WLAN_MAX_APINFO_NUM，表示你需要获得的热点信息个数。扫描完成后会被设定为实际热点个数。

- 举例

```
MI_WLAN_ScanResult_t stRst;  
  
while(1)
```

```

{
    stRst.u8APNumber = 64;

    MI_WLAN_Scan(NULL, &stRst); //默认stRst.u8APNumber=
    MI_WLAN_MAX_APINFO_NUM

    for(i = 0 ; i < stRst.u8APNumber; i ++)
    {

        printf("%s %d %s\n", __FUNCTION__, __LINE__,
stRst.stAPInfo[i].au8SSID);
    }

    memset(&stRst,0,sizeof(stRst));

    stRst.u8APNumber = 64;

    sleep(3);

}

```

2.9. MI_WLAN_GetStatus

- 功能

获取当前所有wifi连接信息

- 语法

```

MI_RESULT MI_WLAN_GetStatus(WLAN_HANDLE hWlan, MI_WLAN_Status_t
*pstStatus);

```

- 形参

参数名称	描述	输入/输出
hWlan	Wlan 句柄	输入
pstStatus	当前所有wifi连接信息	输出

- 返回值

- 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
 - 依赖
 - 头文件: mi_wlan.h mi_wlan_datatype.h
 - 库: libmi_wlan libcjson libcrypto libssl libnl
 - 可执行档: iwlist wpa_cli wpa_supplicant
 - 注意
 - MI_WLAN_Status_t 为一个union，根据当前open时设置的网络类型，返回已经连接的热点信息 或者连接到本设备的主机的信息。
-

2.10. MI_WLAN_GetWlanChipVersion

- 功能
获取wlan设备FW的版本号
- 语法

```
MI_RESULT MI_WLAN_GetWlanChipVersion(MI_U8 *pstChipVersion);
```

- 形参

参数名称	描述	输入/输出
pstChipVersion	wlan设备FW的版本号	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_wlan.h mi_wlan_datatype.h
 - 库: libmi_wlan libcjson libcrypto libssl libnl

- 可执行档: iwlist wpa_cli wpa_supplicant

3. 数据类型

3.1. 模块相关数据类型

数据类型	定义
WLAN_HANDLE [#32-wlan_handle]	定义WLAN句柄
MI_WLAN_Security_e [#33-mi_wlan_security_e]	定义WLAN连接的安全协议类型
MI_WLAN_Encrypt_e [#34-mi_wlan_encrypt_e]	定义WLAN连接的加密协议类型
MI_WLAN_NetworkType_e [#35-mi_wlan_networktype_e]	定义WLAN设备的工作模式类型
MI_WLAN_Authentication_Suite_e [#36-mi_wlan_authentication_suite_e]	定义WLAN连接认证的协议类型
MI_WLAN_WPAStatus_e [#37-mi_wlan_wpastatus_e]	定义WLAN连接的状态类型
MI_WLAN_InitParams_t [#38-mi_wlan_initparams_t]	定义WLAN设备的初始化参数
MI_WLAN_OpenParams_t [#39-mi_wlan_openparams_t]	定义WLAN设备使能的参数
MI_WLAN_ConnectParam_t [#310-mi_wlan_connectparam_t]	定义WLAN连接的参数
MI_WLAN_Quality_t [#311-mi_wlan_quality_t]	定义wifi热点的信号质量
MI_WLAN_Cipher_t [#312-mi_wlan_cipher_t]	定义wifi热点工作的加密信息
MI_WLAN_APInfo_t [#313-mi_wlan_apinfo_t]	定义wifi热点的信息

MI_WLAN_ScanParam_t [#314-mi_wlan_scanparam_t]	定义wifi热点扫描的参数信息
MI_WLAN_ScanResult_t [#315-mi_wlan_scanresult_t]	定义wifi热点扫描的结果信息
MI_WLAN_Status_sta_t [#316-mi_wlan_status_sta_t]	定义正在连接的wifi热点的信息
MI_WLAN_Status_host_t [#317-mi_wlan_status_host_t]	定义某个接入的主机的信息
MI_WLAN_Status_ap_t [#318-mi_wlan_status_ap_t]	定义所有接入的主机的信息
MI_WLAN_Status_t	Union of MI_WLAN_Status_host_t and MI_WLAN_Status_sta_t

3.2. WLAN_HANDLE

- 说明

Wlan设备句柄，标记并区分wifi连接。

- 定义

```
typedef MI_S32 WLAN_HANDLE
```

3.3. MI_WLAN_Security_e

- 说明

Wifi通讯的安全协议标准类型。

- 定义

```
typedef enum
```

```

{

    /// WLAN module security key off

    E_MI_WLAN_SECURITY_NONE          = 1 << 0,

    /// WLAN module security key unknow

    E_MI_WLAN_SECURITY_UNKNOWTYPE = 1 << 1,

    /// WLAN module security key WEP

    E_MI_WLAN_SECURITY_WEP          = 1 << 2,

    /// WLAN module security key WPA

    E_MI_WLAN_SECURITY_WPA          = 1 << 3,

    /// WLAN module security key WPA2

    E_MI_WLAN_SECURITY_WPA2        = 1 << 4,

    /// WLAN module max

    E_MI_WLAN_SECURITY_MAX          = 0xff,

} MI_WLAN_Security_e;

```

3.4. MI_WLAN_Encrypt_e

- 说明

Wifi密钥加密类型。

- 定义

```

typedef enum

{

    /// WLAN module encrypt type none

    E_MI_WLAN_ENCRYPT_NONE          = 1 << 0,

```

```

    /// WLAN module encrypt type unknown

    E_MI_WLAN_ENCRYPT_UNKNOWN    = 1 << 1,

    /// WLAN module encrypt type WEP

    E_MI_WLAN_ENCRYPT_WEP       = 1 << 2,

    /// WLAN module encrypt type TKIP

    E_MI_WLAN_ENCRYPT_TKIP      = 1 << 3,

    /// WLAN module encrypt type AES

    E_MI_WLAN_ENCRYPT_AES       = 1 << 4,

    /// WLAN module max

    E_MI_WLAN_ENCRYPT_MAX       = 0xff,

} MI_WLAN_Encrypt_e;

```

3.5. MI_WLAN_NetworkType_e

- 说明

Wlan设备的工作模式。

- 定义

```

typedef enum

{

    /// WLAN network infrastructure type

    E_MI_WLAN_NETWORKTYPE_INFRA,

    /// WLAN network AP type

    E_MI_WLAN_NETWORKTYPE_AP,

    /// WLAN network AdHoc type

    E_MI_WLAN_NETWORKTYPE_ADHOC,

```

```

    /// WLAN network Monitor type
    E_MI_WLAN_NETWORKTYPE_MONITOR,

    /// WLAN network mode master
    E_MI_WLAN_NETWORKTYPE_MASTER,

    /// WLAN network mode slave
    E_MI_WLAN_NETWORKTYPE_SLAVE,

    /// WLAN param max
    E_MI_WLAN_NETWORKTYPE_MAX

} MI_WLAN_NetworkType_e;

```

- 注意事项

目前支持E_MI_WLAN_NETWORKTYPE_INFRA 和
E_MI_WLAN_NETWORKTYPE_AP

3.6. MI_WLAN_Authentication_Suite_e

- 说明

WIFI认证方式。

- 定义

```

typedef enum
{
    /// Authentication Suite PSK
    E_MI_WLAN_AUTH_SUITE_PSK,

    /// Authentication Suite unknown
    E_MI_WLAN_AUTH_SUITE_UNKNOWN,

    E_MI_WLAN_AUTH_SUITE_MAX
}

```



```
} MI_WLAN_Authentication_Suite_e;
```

3.7. MI_WLAN_WPAStatus_e

- 说明

Wifi连接过程中的状态。

- 定义

```
typedef enum
{
    WPA_DISCONNECTED,
    WPA_INTERFACE_DISABLED,
    WPA_INACTIVE,
    WPA_SCANNING,
    WPA_AUTHENTICATING,
    WPA_ASSOCIATING,
    WPA_ASSOCIATED,
    WPA_4WAY_HANDSHAKE,
    WPA_GROUP_HANDSHAKE,
    WPA_COMPLETED
} MI_WLAN_WPAStatus_e;
```

3.8. MI_WLAN_InitParams_t

- 说明

Wlan设备初始化参数。

- 定义

```
typedef struct MI_WLAN_InitParams_s
{
    /// json description file of wifi
    dongle

    MI_U8 au8JsonConfFilePath[MI_WLAN_MAX_FOLDERPATH_LEN];

    /// reserved

    MI_U64 u64Reserved;
} MI_WLAN_InitParams_t;
```

- 成员

成员名称	描述
au8JsonConfFilePath	Wlan设备初始化依赖的json配置文件绝对路径。
u64Reserved	预留设置，未启用。

- 注意事项

Wlan设备的初始化，依赖一个json配置文件，公版名为wlan.json，位于/config/wifi/wlan.json,该参数目前不支持传入NULL。

3.9. MI_WLAN_OpenParams_t

- 说明

Wlan设备的工作参数。

- 定义

```
typedef struct MI_WLAN_OpenParam_s
{

```

```
// WLAN network type

MI_WLAN_NetworkType_e eNetworkType;

// reserved

MI_BOOL bReserved;

} MI_WLAN_OpenParams_t;
```

- 成员

成员名称	描述
MI_WLAN_NetworkType_e	Wlan 设备的工作服务类型。
bReserved	预留参数，未启用。

- 相关数据类型及接口

[MI_WLAN_NetworkType_e](#) [#35-mi_wlan_networktype_e]

3.10. MI_WLAN_ConnectParam_t

- 说明

Wlan设备连接的具体参数设定。

- 定义

```
typedef struct
{

    // Wlan security mode

    MI_WLAN_Security_e eSecurity;

    // Wlan SSID

    MI_U8 au8SSID[MI_WLAN_MAX_SSID_LEN];
```

```
// Wlan password

MI_U8 au8Password[MI_WLAN_MAX_PASSWD_LEN];

// WLAN connect overtime

MI_U32 OverTimeMs;

} MI_WLAN_ConnectParam_t;
```

- 成员

成员名称	描述
MI_WLAN_Security_e [#33-mi_wlan_security_e]	infra [#35-mi_wlan_networktype_e] 模式下为wifi热点的security type。 ap [#35-mi_wlan_networktype_e] 模式下为接入wifi连接时采用的 security type
au8SSID	wifi热点的名字。
au8Password	wifi 热点的密码
OverTimeMs	建立wifi连接的超时时间

- 注意事项

意见使用WLAN module security key WPA/ WLAN module security key WPA2

- 相关数据类型及接口

[MI_WLAN_NetworkType_e](#) [#35-mi_wlan_networktype_e]

[MI_WLAN_Security_e](#) [#33-mi_wlan_security_e]

3.11. MI_WLAN_Quality_t

- 说明

Wifi热点的信号质量。

- 定义

```
typedef struct
{
    MI_U8 curLVL;

    MI_U8 maxLVL;

    MI_S8 signalSTR;
} MI_WLAN_Quality_t;
```

- 成员

成员名称	描述
curLVL	当前wifi热点的信号水平
maxLVL	Wifi热点的总信号水平
signalSTR	当前wifi热点的信号强度(mdb)

3.12. MI_WLAN_Cipher_t

- 说明

Wifi连接的加密设定

- 定义

```
typedef struct
{
    // WLAN Security mode

    MI_WLAN_Security_e eSecurity;

    // WLAN Encryption type

    MI_WLAN_Encrypt_e eGroupCipher;
```

```

// WLAN Encryption type

MI_WLAN_Encrypt_e ePairCipher;

// WLAN authentication suite

MI_WLAN_Authentication_Suite_e eAuthSuite;

} MI_WLAN_Cipher_t;

```

- 成员

成员名称	描述
eSecurity	Wifi热点的安全协议类型
eGroupCipher	Wifi热点的组加密类型
ePairCipher	Wifi热点的配对加密类型
eAuthSuite	Wifi连接的认证协议组

- 相关数据类型及接口

[MI_WLAN_Security_e](#) [#33-mi_wlan_security_e]

[MI_WLAN_Encrypt_e](#) [#34-mi_wlan_encrypt_e]

[MI_WLAN_Authentication_Suite_e](#) [#36-mi_wlan_authentication_suite_e]

3.13. MI_WLAN_APIInfo_t

- 说明

Wifi热点信息。

- 定义

```

typedef struct MI_WLAN_APIInfo_s
{

```

```

// WLAN CELL ID

MI_U16 u16CellId;

// WLAN Frequency GHz

MI_FLOAT fFrequency;

// WLAN Bitrate Mb/s

MI_FLOAT fBitRate;

// WLAN Quality

MI_WLAN_Quality_t stQuality;

// WLAN Encryption key on/off

MI_BOOL bEncryptKey;

// WLAN SSID

MI_U8 au8SSID[MI_WLAN_MAX_SSID_LEN];

// WLAN Channel

MI_U8 u8Channel;

// WLAN MAC

MI_U8 au8Mac[MI_WLAN_MAX_MAC_LEN];

// WLAN Encryption type

MI_WLAN_Encrypt_e eEncrypt;

// WLAN AP type (Infrastructure /
Ad-Hoc)

MI_WLAN_NetworkType_e eMode;

// WLAN cipher kit

MI_WLAN_Cipher_t stCipher[2];

} MI_WLAN_APInfo_t;

```

- 成员

成员名称	描述
u16CellId	Wifi热点编号
fFrequency	Wifi热点的工作频率
fBitRate	Wifi热点的工作比特率。
stQuality	Wifi热点信息
bEncryptKey	Wifi热点是否需要密码
au8SSID	Wifi热点名字
u8Channel	Wifi热点工作信道
au8Mac	Wifi热点的mac地址
eEncrypt	Wifi热点加密协议
eMode	Wifi热点的工作模式
stCipher	Wifi热点的加密协议组

- 相关数据类型及接口

[MI_WLAN_Quality_t](#) [#311-mi_wlan_quality_t]

[MI_WLAN_Encrypt_e](#) [#34-mi_wlan_encrypt_e]

[MI_WLAN_NetworkType_e](#) [#35-mi_wlan_networktype_e]

[MI_WLAN_Cipher_t](#) [#312-mi_wlan_cipher_t]

3.14. MI_WLAN_ScanParam_t

- 说明

wifi热点扫描设定。

- 定义


```
typedef struct MI_WLAN_ScanParam_s
{
    // Wlan set block mode

    MI_BOOL bBlock; //reserved
} MI_WLAN_ScanParam_t;
```

- 成员

成员名称	描述
bBlock	是否阻塞 未启用，是否阻塞在json配置文件当中配置

3.15. MI_WLAN_ScanResult_t

- 说明

Wifi热点扫描结果。

- 定义

```
typedef struct MI_WLAN_ScanResult_s
{
    // Wlan AP number

    MI_WLAN_APInfo_t stAPInfo
    [MI_WLAN_MAX_APINFO_NUM];

    MI_U8 u8APNumber;
} MI_WLAN_ScanResult_t;
```

- 成员

成员名称	描述
stAPInfo	保留，未使用。
u8APNumber	

- 相关数据类型及接口

[MI_WLAN_APInfo_t](#) [#313-mi_wlan_apinfo_t]

3.16. MI_WLAN_Status_sta_t

- 说明

Wifi连接状态。

- 定义

```
typedef struct MI_WLAN_Status_sta_s
{
    MI_U8 bssid[MI_WLAN_BSSID_LEN];

    MI_U32 freq;

    MI_U8 ssid[MI_WLAN_MAX_SSID_LEN];

    MI_U16 id;

    MI_WLAN_NetworkType_e mode;

    MI_WLAN_Cipher_t stCipher;

    MI_U8 key_mgmt[12];

    MI_WLAN_WPAStatus_e state;

    MI_U8 address[MI_WLAN_BSSID_LEN];

    MI_U8 ip_address[16];

    MI_U32 channel;
```

```
MI_U32 RSSI;  
  
MI_U8 Bandwidth[8];  
  
} MI_WLAN_Status_sta_t;
```

- 成员

成员名称	描述
bssid	Wifi热点名字hexdecimal值
freq	正在连接的wifi热点频率。
ssid	Wifi热点名字。
id	Wifi热点id
mode	Wifi热点工作模式。
stCipher	Wifi热点加密协议组。
key_mgmt	Wifi热点密钥管理类型。
state	Wifi热点当前的连接状态
address	Wifi热点给主机分配的ip地址
channel	当前wifi连接工作信道
RSSI	当前wifi连接的RSSI水平
Bandwidth	当前wifi连接工作带宽

- 相关数据类型及接口

[MI_WLAN_NetworkType_e](#) [#35-mi_wlan_networktype_e]

[MI_WLAN_Cipher_t](#) [#312-mi_wlan_cipher_t]

[MI_WLAN_WPAStatus_e](#) [#37-mi_wlan_wpastatus_e]

3.17. MI_WLAN_Status_host_t

- 说明

Ap模式下接入wifi连接的状态信息。

- 定义

```
typedef struct MI_WLAN_Status_host_s
{
    MI_U8 hostname          [MI_WLAN_MAX_HOST_NAME_LEN];

    MI_U8 ipaddr[16];

    MI_U8 macaddr[18];

    MI_U64 connectedtime;

} MI_WLAN_Status_host_t;
```

- 成员

成员名称	描述
hostname	接入主机的名字。
ipaddr	接入主机的ip地址。
macaddr	接入主机的mac地址
connectedtime	接入主机的已连接时间

3.18. MI_WLAN_Status_ap_t

- 说明

所有接入wifi接连的状态信息。

- 定义

```
typedef union MI_WLAN_Status_s
{
    MI_WLAN_Status_sta_t stStaStatus;

    MI_WLAN_Status_ap_t stApStatus;
} MI_WLAN_Status_t;
```

- 成员

成员名称	描述
stStaStatus	当前连接的wifi热点信息
stApStatus	当前接入的所有主机的信息

- 相关数据类型及接口

[MI_WLAN_Status_sta_t](#) [#316-mi_wlan_status_sta_t]

[MI_WLAN_Status_ap_t](#) [#318-mi_wlan_status_ap_t]

4. 错误码

宏定义	描述
MI_WLAN_ERR_FAIL	未识别错误
MI_WLAN_ERR_INVALID_DEVID	音频输入信道号无效
MI_WLAN_ERR_ILLEGAL_PARAM	输入参数不合法
MI_WLAN_ERR_NOT_SUPPORT	输入参数为不支持类型
MI_WLAN_ERR_MOD_INITED	设备已经初始化
MI_WLAN_ERR_MOD_NOT_INIT	设备未初始化
MI_WLAN_ERR_NOT_CONFIG	设备未使能
MI_WLAN_ERR_INVAL_HANDLE	句柄非法
MI_WLAN_ERR_NULL_PTR	空输入参数
MI_WLAN_ERR_INITED	模块已经初始化

5. APPENDIX A

wlan.json:

```
{
    /*当前选中的wifi设备，本配置文件支持多个wifi设配的配置
    **本例选择 ssw01b,那么MI_WLAN就会去解析ssw01b标签下的信息
    */
    "selfie": {
        "wifi": "ssw01b"
    },
    /*wifi设备标签*/
    "ssw01b": {
        /*wifi 设备的自我描述*/
        "selfie": {
            /*wifi设备的固件版本*/
            "version": "0.0.0",
```

```

/*wifi设备的名字*/
"name": "ssw01b40m",
/*wifi设备初始化脚本的路径，会在MI_WLAN_Init中被调用*/
"scriptPath": "/config/wifi",
/*本wifi设备所有script标签下的脚本解析缺省解析bin
**脚本不一定需要是shell，也可以是python perl等
*/
"parser": "sh"
},
/*wifi的初始化标签
**在MI_WLAN_Init函数中被解析执行
*/
"init": {
/*本wifi设备运行需要配置的linux环境变量
**环境变量的name与value需要一一对应
**个数不限
*/
"env": {
"name": ["LD_LIBRARY_PATH", "PATH"],
"value": ["/config/wifi", "/config/wifi"]
},
/*本wifi设备初始化需要解析的脚本
**@parser 解析这个脚本的bin档，可选标签
**@name 脚本文件
**@option 脚本参数，个数不限，可选标签
*/
"script": {
"parser": "source",
"name": "ssw01bInit.sh",
"option": ["dummy1", "dummy2"]
}
},
/*wifi设备的注销标签*/
"deinit": {
/*本wifi设备注销需要解析的脚本
**@parser 可选标签，未选
**@name 脚本文件
**@option 可选标签，未选
*/
"script": {
"name": "ssw01bDeInit.sh"
}
},
/*wifi设备的全局信息配置标签
**本标签给出了一些wifi服务需要的基本设定
**用户可以根据自身对wifi工作的定制需求，
**配置该标签下的内容
*/
"universal": {

```

```

/*主机模式下配置标签*/
"infra": {
    /*wifi设备提供的wifi热点的接口名字
    **不同wifi设备提供的接口名字可能会有差异
    */
    "iface": "wlan0",
    /*系统提供给wifi设备的控制接口目录
    **需要在init之前创建好，默认是在init标签中指示的脚本中创建
    **需要用户同步路径
    */
    "ctrl_interface": "/config/wifi/run/wpa_supplicant"
},
/*热点模式下的配置标签*/
"ap": {
    /*wifi设备提供的wifi热点的接口名字
    **不同wifi设备提供的接口名字可能会有差异
    */
    "iface": "p2p0",
    /*热点自己的static ip地址*/
    "ipaddr": "192.168.1.100",
    /*热点自己的子网掩码*/
    "netmask": "255.255.255.0",
    /*系统提供给wifi设备的控制接口目录
    **需要在init之前创建好，默认是在init标签中指示的脚本中创建
    **需要用户同步路径
    */
    "ctrl_interface": "/var/run/hostapd"
},
/*udhcpd 需要的配置脚本路径，该脚本用于配置获取的网络连接信息，系统提供*/
"dhcp_script": "/etc/init.d/udhcpd.script",
/*wpa_supplicant 需要的配置文件路径，需要用户构建*/
"wpa_supplicant_conf":
"/config/wifi/wpa_supplicant.conf",
/*hostapd 需要的配置文件路径，需要用户构建*/
"hostapd_conf": "/config/wifi/hostapd.conf",
/*dnsmasq 需要的配置文件路径，需要用户构建*/
"dnsmasq_conf": "/config/wifi/dnsmasq.conf",
/*dhcp 需要的租约记录文件，需要用户构建目录*/
"dhcp-leasefile": "/var/lib/misc/dnsmasq.leases"
},
/*
**individual 标签定义了MI_WLAN 支持的所有action
**action子标签 目前总共定义了
**@scan 扫描操作
**@open 打开WLAN设备操作
**@close 关闭WLAN设备操作
**@connect wifi连接服务

```


****@disconnect** wifi连接服务断开
****@status** 获取当前wifi连接服务状态
******每一个action在不同的工作模式下会有不同的行为
****action** 定义模式为

```
**{  
    "binary":["bin1",...."binx"],  
    "option0":["opt1",...."optX"],  
        .....  
        .....  
        .....  
    "option#N":["opt1",...."optX"]  
**}
```

****binary**数组的bin和option数组是一一对应的，option标签有明确的编号尾缀，从0开始递增

****option**内容，支持对universal 标签的内容引用，语法为'\$TAGNAME'支持逐级引用

```
**'$TAGNAME:$SUB_TAGNAME'  
}  
*/  
"individual": {  
    "action": {  
        "scan": {  
            "binary": ["iwlist"],  
            "option0": ["$infra:$iface", "scanning"]  
        },  
        "open": {  
            "deviceup": {  
                "ap": {  
                    "binary": ["ifconfig"],  
                    "option0": ["$ap:$iface", "up"]  
                },  
                "infra": {  
                    "binary": ["ifconfig"],  
                    "option0": ["$infra:$iface", "up"]  
                }  
            },  
            "serviceup": {  
                "ap": {  
                    "binary": ["ifconfig", "hostapd"],  
                    "option0":  
["$ap:$iface", "$ap:$ipaddr", "netmask", "$ap:$netmask"],  
                    "option1": ["-B", "$hostapd_conf"]  
                },  
                "infra": {  
                    "binary": ["wpa_supplicant"],  
                    "option0": ["-i", "$infra:$iface", "-Dnl80211", "-c", "$wpa_supplicant_conf", "-d", "&"]  
                }  
            }  
        }  
    }  
}
```

```

    },
    "close": {
        "devicedown": {
            "ap": {
                "binary": ["ifconfig"],
                "option0": ["$ap:$iface", "down"]
            },
            "infra": {
                "binary": ["ifconfig"],
                "option0": ["$infra:$iface", "down"]
            }
        },
        "servicedown": {
            "ap": {
                "script": {
                    "parser": "sh",
                    "name": "atbmClose.sh",
                    "option": ["ap"]
                }
            },
            "infra": {
                "script": {
                    "name": "atbmClose.sh",
                    "option": ["infra"]
                }
            }
        }
    },
    "connect": {
        "ap": {
            "binary": ["ifconfig", "dnsmasq"],
            "option0": ["$ap:$iface", "up"],
            "option1": ["-i", "$ap:$iface", "--no-daemon", "-C", "$dnsmasq_conf", "&"]
        },
        "infra": {
            "add": {
                "binary": ["wpa_cli"],
                "option0": ["-i", "$infra:$iface", "-p", "$infra:$ctrl_interface", "add_network"]
            },
            "setup": {
                "ssid": {
                    "binary": ["wpa_cli"],
                    "option0": ["-i", "$infra:$iface", "-p", "$infra:$ctrl_interface", "set_network", "%d", "ssid", "\\\\"$s\\\\""]
                },
                "password": {

```

```

        "keyon":{
            "wpa":{
                "binary":["wpa_cli"],
                "option0": ["-
i","$infra:$iface","-
p","$infra:$ctrl_interface","set_network","%d","psk","\\\\"%s\\\\""]
            },
            "wep":{
                "binary":["wpa_cli"],
                "option0": ["-
i","$infra:$iface","-
p","$infra:$ctrl_interface","set_network","%d","wep_key0","\\\\"%s\\\\""]
            }
        },
        "keyoff":{
            "binary":["wpa_cli"],
            "option0": ["-i","$infra:$iface","-
p","$infra:$ctrl_interface","set_network","%d","key_mgmt","NONE"]
        }
    },
    "enable": {
        "binary": ["wpa_cli","wpa_cli"],
        "option0": ["-i","$infra:$iface","-
p","$infra:$ctrl_interface","select_network","%d"],
        "option1": ["-i","$infra:$iface","-
p","$infra:$ctrl_interface","enable_network","%d"]
    },
    "dhcpc": {
        "binary": ["udhcpc"],
        "option0": ["-i","$infra:$iface","-
s","$dhcp_script","-a","-q","-b","-t","20","-T","1"]
    },
    "save": {
        "binary": ["wpa_cli"],
        "option0": ["-i","$infra:$iface","-
p","$infra:$ctrl_interface","save_config"]
    }
},
"disconnect": {
    "ap": {
        "binary": ["ifconfig"],
        "option0": ["$ap:$iface","down"]
    },
    "infra": {
        "binary": ["wpa_cli"],

```

```

        "option0": ["-i ", "$infra:$iface", "-p", "$infra:$ctrl_interface", "disable_network", "%d"]
    },
    "status": {
        "ap": {
            "all_sta": {
                "binary": ["hostapd_cli"],
                "option0": ["-i", "$ap:$iface", "-p", "$ap:$ctrl_interface", "all_sta"]
            }
        },
        "infra": {
            "binary": ["wpa_cli"],
            "option0": ["-i", "$infra:$iface", "-p", "$infra:$ctrl_interface", "status"]
        }
    }
}
}
}
}
}
}
}

```